



ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ ГОРОДА МОСКВЫ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ГОРОДА МОСКВЫ
«КОЛЛЕДЖ СВЯЗИ №54»
ИМЕНИ П.М. ВОСТРУХИНА

Дисциплина: Информационные системы и программирование

Отчет по выполнению задания на тему: «Разработка
подсистемы безопасности**».**

Выполнил:

Группа: **4ИСП9-45**

Студент: **Маматисаков Элмурат
Сапарбекович**

Проверил преподаватель:

С. Клименко

Москва 2026 г.

Практическая работа № 4

Разработка подсистемы безопасности

Выполненные изменения

Механизм	Реализация
Пароли	Werkzeug, PBKDF2 SHA256, 600000 итераций и случайная соль. В базе сохраняется хеш.
SQL запросы	Параметры ? используются для всех пользовательских значений.
Роли	login_required, participant_required, admin_required проверяют доступ. Роль берется из базы.
Валидация	Белые списки и ограничения на клиенте и сервере. Проверяются ФИО, телефон, email, дата, время, оплата и оценка.
Сессии	Случайный SECRET_KEY из .env. HttpOnly, SameSite=Lax, 30 минут бездействия. При входе session.clear().
Выход и CSRF	Выход меняет версию сессии; прежняя cookie теряет доступ. Изменяющие запросы требуют CSRF токен.
Вывод текста	Шаблоны экранируют HTML. Отзыв с тегами не выполняется как JavaScript.

Флаг Secure отключен для локального HTTP. При запуске через HTTPS он включается настройкой COOKIE_SECURE=1. .env не попадает в репозиторий; ключ не выводится в отчете.

Проверка защиты

```
Конференции РФ - проверка X + | v
PS> python -m unittest discover -s tests
.....
-----
Ran 22 tests in 4.643s

OK

PS> python test_hash.py
Алгоритм: pbkdf2:sha256:600000
Хеш: pbkdf2:sha256:600000$prze5jr6DuXkhTxwi$7df318e809fc31aa8b3b1e7aa04b8cfe2ab4395cf0a49189d619fed1abf2c786
Верный пароль: True
Неверный пароль: False

PS> python test_sql_injection.py
Подготовленный запрос с параметром ?
Результат инъекции: 0 записей

PS> python security_summary.py
Вход администратора: HTTP 200
Cookie conf_session: HttpOnly = True
Cookie conf_session: SameSite=Lax = True
Secure для локального HTTP = False
Время бездействия до выхода: 30 минут
SECRET_KEY: загружен из окружения; значение не выводится
Для HTTPS COOKIE_SECURE=1 включает Secure
Нажмите Enter после снимка:
```

Рисунок 1. Хеширование, SQL инъекция, cookies и 22 проверки

Проверка	Результат
Верный / неверный пароль	True / False при проверке хеша.
Логин с SQL инъекцией	Ноль записей; вход отклонен, HTTP 401.
Гость / участник в админке	Гость перенаправляется на вход; участник получает HTTP 403.
Чужая заявка / повторный отзыв	Чужая заявка не отображается; операция с ней и второй отзыв отклоняются.
Сессия и CSRF	Подмененная, просроченная и старая cookie после выхода отклонены. Запрос без CSRF токена отклонен.

Проверка в браузере

Конференции.РФ Бронирование помещений elmurat2027 Выйти

Регистрация

Заполните все поля, чтобы отправлять заявки на помещение.

Логин
abc

Пароль

ФИО
Иванов Иван Иванович

Телефон
8(999)123-45-67

Email
student@example.ru

Логин: от 6 до 50 символов, латиница и цифры

Создать пользователя

[Уже зарегистрированы? Войти](#)

Рисунок 2. Клиентская проверка некорректного логина

В живом браузере cookie `conf_session` имела `HttpOnly=true` и `SameSite=Lax`. Участнику запрещен доступ к `/admin`. Серверная проверка действует и при обходе JavaScript: неправильные логин, пароль, ФИО, email и телефон дают HTTP 400.

В Git отдельно сохранены параметры SQL, хеширование, валидаторы, роли, защита сессий и клиентские формы. Сделано более пяти коммитов. Репозиторий: <https://github.com/Cla1ve/conference-portal-practices-2026-10-01>.

Контрольные вопросы

1. Почему пароли нельзя хранить открыто или шифровать?

Открытые пароли читаются при утечке базы. Шифрование обратимо при наличии ключа. Для проверки паролей нужен медленный хеш с солью.

2. Как параметры защищают от SQL инъекций?

SQL команда и значения передаются отдельно. Кавычки во вводе остаются частью строки, а не меняют запрос.

3. Чем отличаются проверки клиента и сервера?

Клиент улучшает удобство, но его можно обойти. Сервер обязательно проверяет данные перед сохранением.

4. Зачем HttpOnly?

Он запрещает JavaScript читать session cookie через document.cookie.

5. Что такое CSRF?

Это выполнение нежелательного действия от имени вошедшего пользователя. В проекте проверяется случайный токен и установлен SameSite=Lax.

6. Почему ключ хранится в окружении?

Чтобы секрет для подписи сессий не оказался в открытом коде. Файл .env включен в .gitignore.

7. Как разграничиваются роли?

Гость не работает с заявками. Участник управляет своими заявками и отзывами. Только администратор видит все заявки и меняет статус.

Вывод

Реализованы хеширование, параметры SQL, проверка ролей и данных, защита сессий и CSRF. Все 22 автоматические проверки завершились успешно.